Christopher Hernandez

Operating Systems & Security CYBI 3345 – CSCI 4334

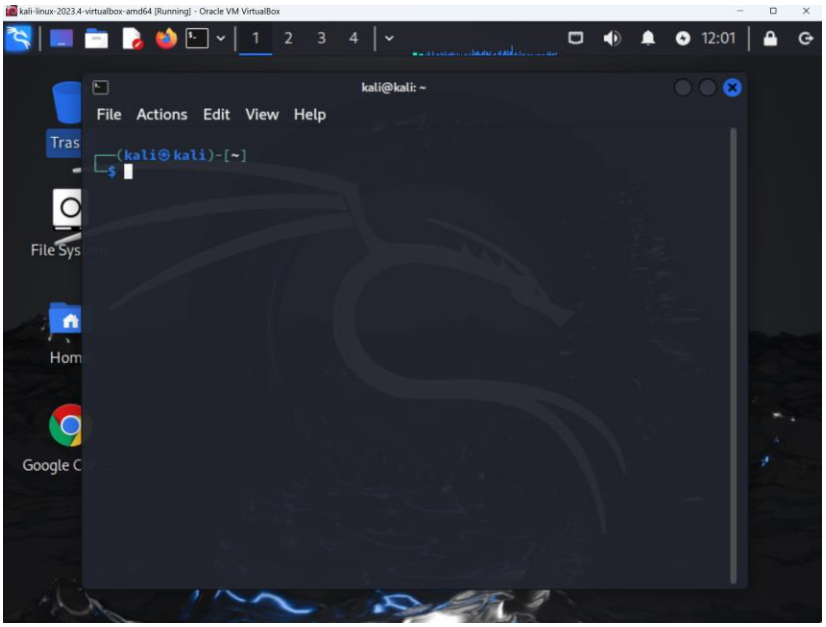
Lab 02- Threads and concurrency in Linux

Introduction: In this lab, I will have hands on practice with the linux system on threads.

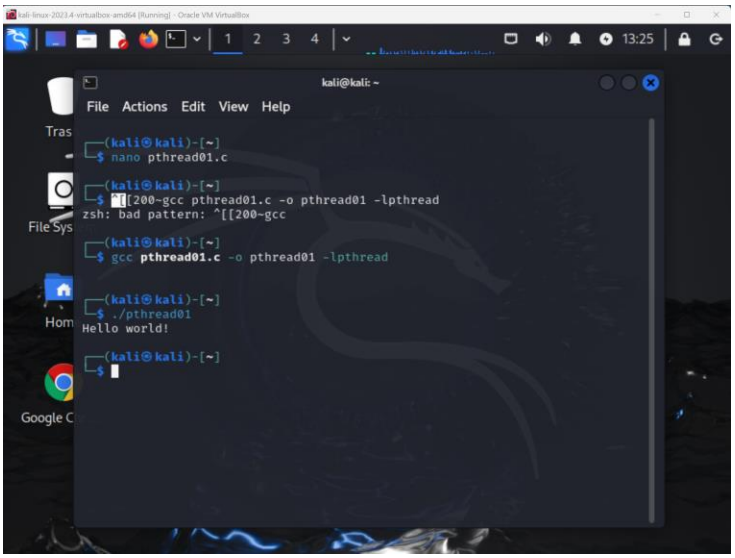
Objective: To manage threads and concurrency in Linux

Step 1:

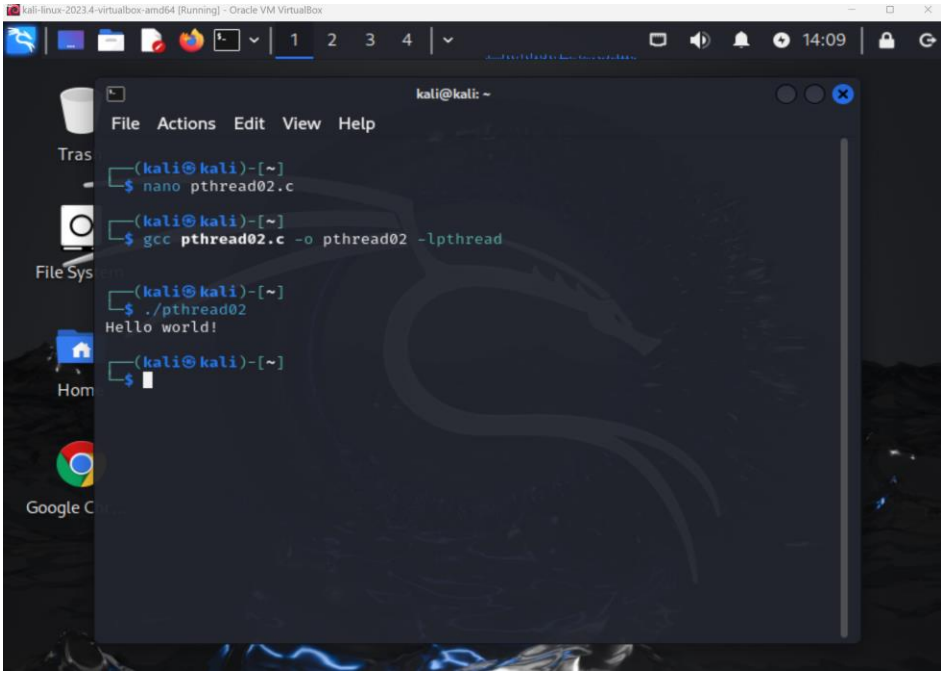
In this step, I simply opened a terminal.



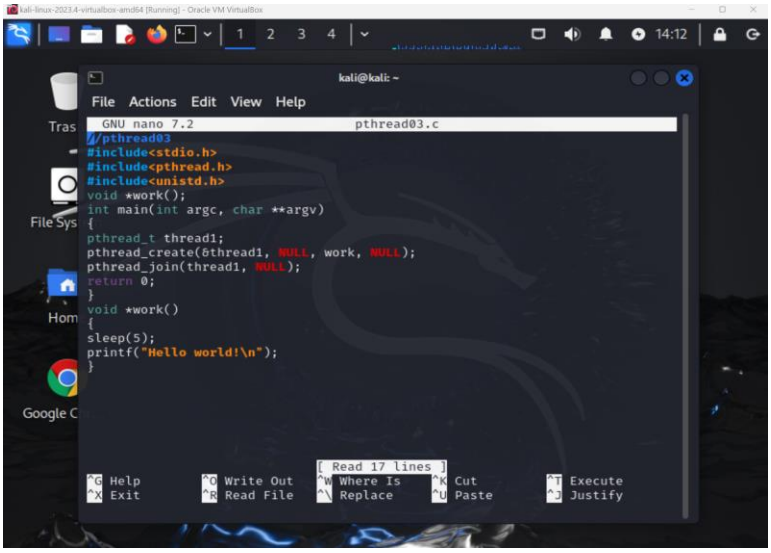
Step 2: In this step, I compiled and ran a code that printed “hello world”



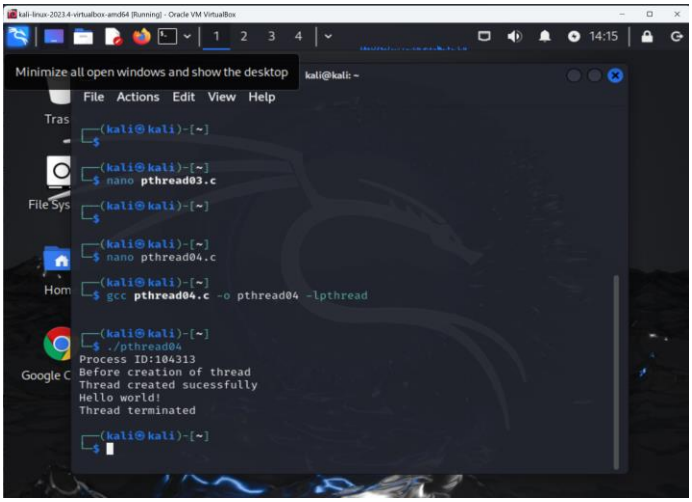
Step 3: I added the sleep function to the code, after running it, hell world still can be seen.

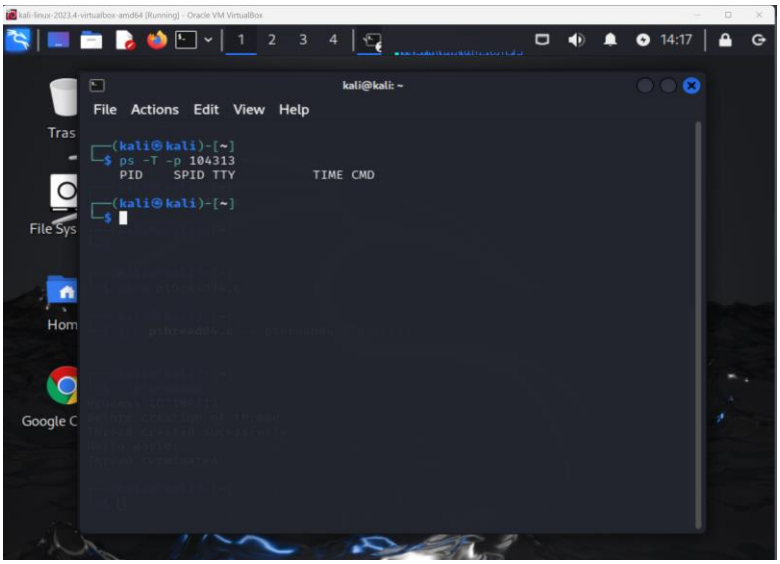


Step 4: I opened the thread and added the code as asked.

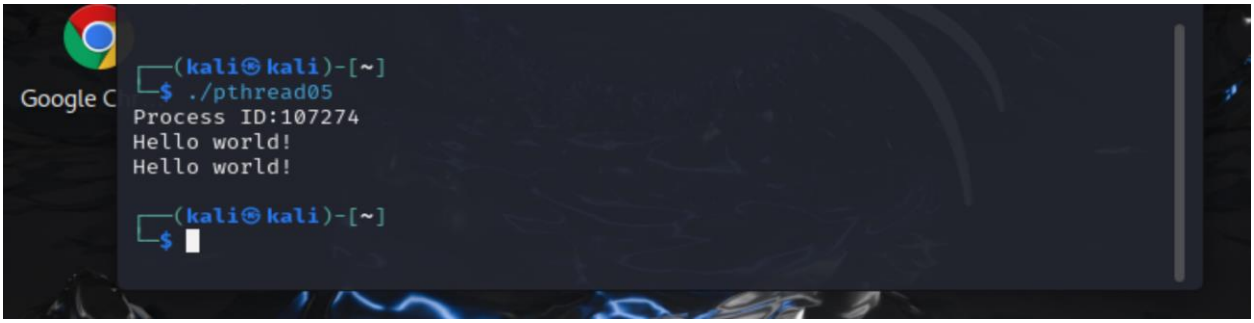


Step 5: I got the process ID.

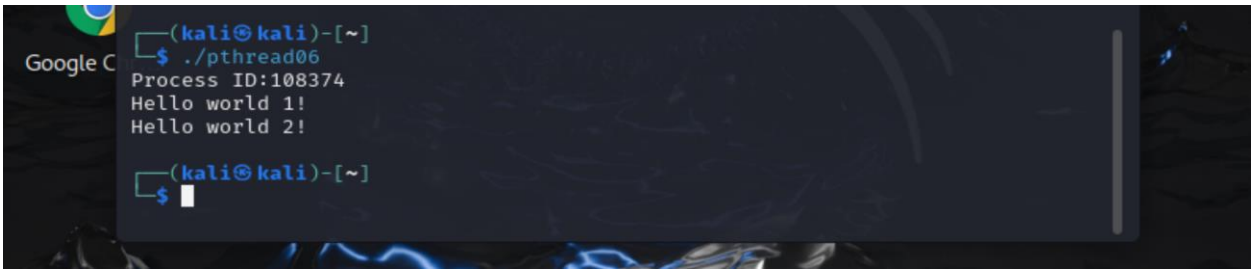




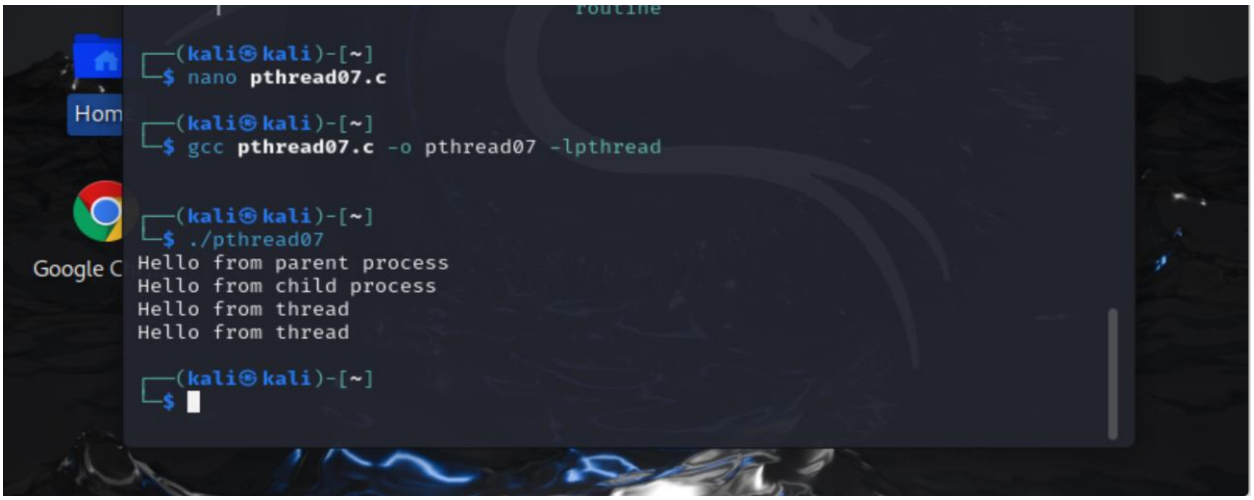
Step 6: I also got the thread ID on this one.



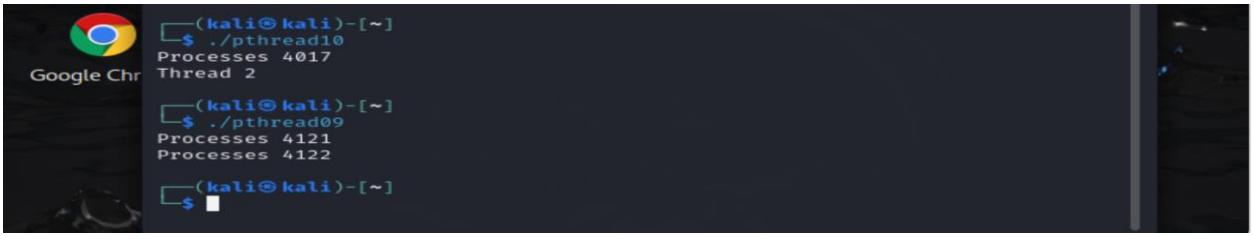
Step 7: I created another function so it can execution its own, but I believe I made a mistake somewhere



Step 8: For this step, I created the child and parent process, I complied and ran them



Step 9: For this step, I got the process ID from both threads by using the provided code.



Step 10: After seeing the provided code, I added x as variable and it increments.

```
┌──(kali㉿kali)-[~]
└─$ ./pthread11
Process id: 4871
Value of x: 2
Process id: 4872
Value of x: 3

┌──(kali㉿kali)-[~]
└─$
```

Step 11: Once changed, the result is very similar to the previous step.

```
┌──(kali㉿kali)-[~]
└─$ ./pthread12
Process id: 6429
Value of x: 2
Process id: 6430
Value of x: 3

┌──(kali㉿kali)-[~]
└─$
```

Step 12: In both routines the value is the same.

```
┌──(kali㉿kali)-[~]
└─$ ./pthread13
Value of x in routine2: 3
Value of x in routine: 3

┌──(kali㉿kali)-[~]
└─$
```

Step 13: After using the provided code, I did the mail counting function.

```
┌──(kali㉿kali)-[~]
└─$ ./pthread14
Number of emails: 2
```

Step 14: I increased the number.

```
┌──(kali㉿kali)-[~]
└─$ ./pthread15
Number of emails: 200
```

Step 15: Once again, I increased the number of mail.

```
┌──(kali㉿kali)-[~]
└─$ nano pthread016.c

┌──(kali㉿kali)-[~]
└─$ gcc pthread016.c -o pthread16 -lpthread

┌──(kali㉿kali)-[~]
└─$ ./pthread16
Number of emails: 2000000

┌──(kali㉿kali)-[~]
└─$
```

Step 16: The result is like step 16.

```
┌──(kali㉿kali)-[~]
└─$ nano pthread017.c

┌──(kali㉿kali)-[~]
└─$ gcc pthread017.c -o pthread17 -lpthread

┌──(kali㉿kali)-[~]
└─$ ./pthread17
Number of mails: 2000000

┌──(kali㉿kali)-[~]
└─$
```

Step 17: The thread totals to 4m.

```
┌──(kali㉿kali)-[~]
└─$ gcc pthread018.c -o pthread18 -lpthread

┌──(kali㉿kali)-[~]
└─$ ./pthread18
Number of mails: 4000000

┌──(kali㉿kali)-[~]
└─$
```

Step 18: There has been a notorious increase in the mails

```
┌──(kali㉿kali)-[~]
└─$ ./pthread19
Number of mails: 1963887

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ gcc pthread020.c -o pthread20 -lpthread

┌──(kali㉿kali)-[~]
└─$ ./pthread20
Hello from thread 0
Hello from thread 1
Hello from thread 3
Hello from thread 2

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ gcc pthread021.c -o pthread21 -lpthread

┌──(kali㉿kali)-[~]
└─$ ./pthread21
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ ./pthread21
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ ./pthread22
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3

┌──(kali㉿kali)-[~]
└─$
```

Steps 21,22,23 all have a very similar output, they are different ways to complete the same task

Questions:

1) Processes have separate memory spaces, so each process has its own copy of the data. Processes communicate through inter-process communication (IPC) mechanisms like pipes, sockets, or shared memory segments. Threads, on the other hand, share the same memory space within a process. This means that all threads within a process can access the same variables and data structures directly. However, this also means that threads must coordinate access to shared resources to avoid race conditions and ensure data consistency.

2) pthread_create is used to create a new thread within a process. It takes several arguments, including a pointer to a pthread_t object that will store the thread ID, thread attributes (usually NULL for default

attributes), a function pointer to the function the thread will execute, and an optional argument to pass to the thread function.

3) In concurrent programming, a critical section is a section of code that accesses shared resources (e.g., variables, data structures) that must not be concurrently accessed by other threads. Only one thread can execute the critical section at a time, ensuring data consistency and avoiding race conditions.

4) A race condition occurs in concurrent programming when the outcome of a program depends on the sequence or timing of uncontrollable events (such as the execution order of threads), which can lead to unpredictable behavior and erroneous results.

5) The code from step 15 did not result in a total count of 2,000,000 due to a lack of synchronization. Multiple threads were incrementing the mails variable concurrently without any protection mechanism, leading to a race condition

6) Mutex (short for mutual exclusion) is a synchronization primitive that allows only one thread to access a resource at a time. It provides exclusive access to the resource and is typically used to protect critical sections. Semaphore is a more general synchronization primitive that can be used to control access to a resource by multiple threads. It can have an initial value greater than 1, allowing a specified number of threads to access the resource concurrently.

7) sem_wait decrements the semaphore value. If the value is greater than zero, it decrements the value and continues. If the value is zero, the call blocks until the value becomes greater than zero. sem_post increments the semaphore value. If there are threads waiting for the semaphore, one of them will be unblocked.

8) I think they are different ways to complete the same task.

Conclusion: This lab helped me understand many things about threads, processes and the linux system in general. I used the provided code to see different things that Linux is capable of.

GeeksforGeeks. (2022, December 7). Processes in linux/unix. GeeksforGeeks. https://www.geeksforgeeks.org/processes-in-linuxunix/

Jun, W. by: C. M. (2024, January 3). Linux process states. Baeldung on Linux. https://www.baeldung.com/linux/process-states#:~:text=In%20Linux%2C%20a%20process%20is,Uninterruptible%20Sleep%20(D)

Wikimedia Foundation. (2024, February 12). Linux. Wikipedia. https://en.wikipedia.org/wiki/Linux